

Slack Architecture

Slack System Design

👤 Neo Kim included in [Deep Dive](#)

📅 2023-05-28 ✍️ 7928 words ⌚ 38 minutes



The target audience for this article falls into the following roles:

- Tech workers
- Students
- Engineering managers

The prerequisite to reading this article is fundamental knowledge of system design components. This article does not cover an in-depth guide on individual system design components.

Disclaimer: The system design questions are subjective. This article is written based on the research I have done on the topic and might differ from real-world implementations. Feel free to share your feedback and ask questions in the comments.

You can read the TL;DR on [Cheatsheet](#). The system design of Slack depends on the design of the [Real-Time Platform](#) and [Real-Time Presence Platform](#). I highly recommend reading the related articles to improve your system design skills.

Download my system design playbook for free on newsletter signup:



The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 • Over 235,000 subscribers

amanpunetha08@gmail.com



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#) and [our Information collection notice](#)

substack

What is Slack?

Slack is a real-time messaging platform used by organizations for communication between team members ¹. Slack can be considered a hybrid between E-Mail and Internet Relay Chat (IRC) because of the persistence of the chat messages ².

At a high level, [Slack](#) is organized into workspaces, where a user can interact with oneself. Each workspace can contain multiple Slack channels and every Slack channel could accommodate multiple users. A user is allowed to join multiple workspaces. The user can send chat messages inside the Slack channels. A Slack channel is a group of users interested in a specific topic. The users are also allowed to share media files and search for particular content.



Watch on

What is Slack?; Source: video by [Slack](#) from [YouTube](#)

At its inception, Slack customers were from small organizations with a few hundred users. The software architecture of Slack had to evolve significantly to scale for supporting customers from giant organizations. Slack sends billions of messages daily across millions of channels in real-time across the globe. The typical traffic pattern of Slack shows that Slack users are active between 9 hours and 17 hours on weekdays while the peak traffic is between 11 hours and 14 hours ³.

Terminology

The following terminology might be helpful for you:

- Data replication: a technique of storing multiple copies of the same data on different nodes to improve the availability and durability of the system
- Data partitioning: a technique of distributing data across multiple nodes to improve the performance and scalability of the system
- Fault tolerance: the ability of a service to recover from a failure without losing data
- High availability: the ability of a service to remain reachable and not lose data even when a failure has occurred

Questions to ask the Interviewer

You should discuss the system requirements with your interviewer to narrow down the use cases of the system as the time window is limited ^{4, 5, 6}.

| Candidate

1. Should the chat service support both one-on-one and group chat?
2. What is the count of daily active users (**DAU**) and the read: write ratio?
3. What is the anticipated number of concurrent users?
4. How many users can you include in a Slack channel?
5. How many users can you include in a Slack workspace?
6. Should Slack support multiple types of clients?
7. Should the chat history be persisted?
8. Is the Slack user base globally distributed?
9. How many organizations do you expect to use Slack?
10. What is the anticipated traffic pattern?

| Interviewer

1. Yes, the chat service should support both one-on-one and group chat functionality in real-time
2. The DAU is 10 million and the read: write ratio is 10: 1
3. Approximately 7 million simultaneously connected users
4. At most 10,000 users can be included in a Slack channel
5. At most 200 thousand users can be included in a Slack workspace

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

8. Yes, there will be a globally distributed user base with more than half the traffic coming from outside the United States
9. You can assume that around 500 thousand organizations will use Slack
10. 2 hours per weekday for an active user and 10 hours per weekday for every client connection

Requirements

The system requirements can be summarized as follows.

| Functional Requirements

- The user can send real-time messages to another user and within a channel (group messaging)
- The chat messages should be persisted and displayed in chronological order to keep the total order of messages
- The user can send media files such as pictures or short audio files
- The chat service should display the online presence status of the users
- A chat message must be received by a user at most once to preserve message integrity

| Non-Functional Requirements

- Low latency
- Highly reliable
- Highly available

Slack API

Slack Application Programming Interface (**API**) can be classified into [web API](#) and [real-time API](#). The web API provides Hypertext Transfer Protocol (**HTTP**) API endpoints to send messages, execute user login, and perform actions such as creating a new Slack channel ⁷. The OAuth token can be used for secure user authorization on private Slack channels ⁸.

The real-time API provides [WebSockets](#) for latency-sensitive full duplex communication between the client and the server. The real-time API can be used for typing indicators, chat message delivery, and displaying [online presence status](#) ⁹.

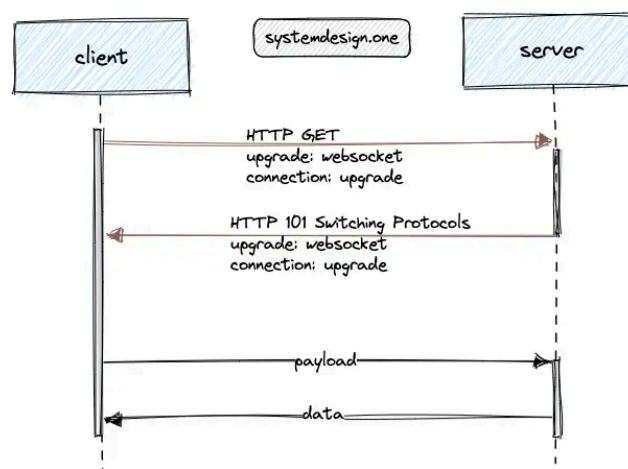


Figure 1: Slack; WebSocket connection through real-time API

The WebSocket protocol keeps the communication channel open for active users. The client performs a protocol switch request to upgrade the client connection to a WebSocket connection. Slack offers dedicated real-time API endpoints for multiple

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

- potential compatibility issues with the existing web infrastructure
- web components such as firewalls and load balancers are built and maintained around HTTP

Although HTTP/2 is not a replacement for push technologies such as WebSocket, combining HTTP/2 with Server-Sent Events (**SSE**) offers an efficient HTTP-based bidirectional communication channel. HTTP/2 allows multiplexing through the reuse of the same TCP connection for multiple SSE streams and several client requests ^{10, 2}.

| How does the client publish a chat message on Slack?

The client executes an HTTP POST request to the web API to send a chat message on Slack ⁷.

▼ JavaScript



```
1 /chat/:channel-id
2 content-type: application/json
3 authorization: Bearer <xyz>
4 method: POST
5
6 {
7   "message": "Breaking Bad can be considered as the greatest TV show",
8   "attachments": [{"text": "Breaking Bad set an unprecedented bar for dra
9 } }
```

The server responds with status code 200 OK on success. The server includes the timestamp of the message in the response and the links will be sanitized.

▼ JavaScript



```
1 status code: 200 OK
2 content-encoding: gzip
3 content-type: application/json
4
5 {
6   "channel-id": "XYZabcd123",
7   "ts": "2893675999.000223",
8   "message": {
9     "text": "Breaking Bad",
10    "user": "Jessie",
11    "attachments": [
12      {
13        "text": "Here is an attachment",
14        "id": 41,
15      }
16    ],
17    "type": "message"
18  }
19 }
```

| How does the Slack client fetch the initial screen?

The client performs an HTTP POST request to the web API for login on Slack. The server returns the snapshot of the entire workspace to display the initial screen on the Slack client ⁵. The snapshot is a keyframe of the team state such as the users, profiles, channels, and membership ².

The client should perform lazy loading of the workspace by fetching the snapshot partially to improve the latency. The server response should also include the WebSocket URL to the real-time API to establish a WebSocket connection with the client ^{4, 11}.

▼ JavaScript



System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

```

4
5 {
6   "url": "wss://ms42.slack-msgs.com/websocket/8I3yRmcfb", //websocket
7   "self": {
8     "user": 52135,
9     "name": "Walter White"
10  },
11  "channels": [{...}], //snapshot
12  "team": {...},
13  "users": [...],
14 }

```

| How does the Slack client display typing indicators?

The real-time API makes use of the WebSocket channel to display the typing indicators ⁷.

✓ JavaScript



```

1 {
2   "user": 42,
3   "type": "typing",
4   "channel": "FB23AVK456"
5 }

```

| API Design Principles

The following API design principles can be used to build a reliable API ^{12, 8}:

- follow the single responsibility principle to build the API and paginate the API response by default
- simplify the API by reducing the number of API parameters and define default parameter values on heavily used API
- provide consistent API endpoints through naming conventions, input parameters, and output responses
- include meaningful error messages in the API response for the ease of troubleshooting
- [rate limit](#) the API and design for [scale](#)
- avoid breaking changes to the API

The [JSON schema](#) validation of the API must be performed through automated tests for increased reliability as the API evolves ⁸.

| Types of Pagination

Slack API must be paginated for [scalability](#). Slack clients should fetch only a limited number of chat messages that fit the viewport of the client while scrolling through the chat history to improve latency and reduce unnecessary bandwidth usage. The different strategies to paginate the API are the following ¹³:

- offset pagination
- cursor-based pagination

| Offset Pagination

The most popular pagination technique is the offset pagination. The client must provide an API request parameter to denote the number of results per page and the page number. The offset pagination performs the following operations at a high-level ¹³:

1. The database is queried to fetch the total count of all the results to determine the total number of pages
2. The database is queried with the count and page number parameters provided in the API request to fetch the items for the requested page

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

SQL



```
1 SELECT COUNT(*) AS total
2 FROM chat
3 WHERE channel_id = %channel_id;
```

The count and page number parameters in the API request are used to query the items from the database for the requested page. The following SQL will query the database with the API request parameters *page number* = 3 and *count* = 20 to return the next 20 items by skipping the initial 40 items:

SQL



```
1 SELECT *
2 FROM chat
3 WHERE channel_id = %channel_id
4 ORDER BY created_at DESC
5 LIMIT 20 OFFSET 40;
```

The server response contains the following data schema:

JavaScript



```
1 {
2   "messages": [...],
3   "paging": {
4     "total": 300, //total count of items on all pages
5     "page": 3,   //current page number
6     "pages": 10  //total number of pages available
7   }
8 }
```

The advantages of using offset pagination are the following ¹³:

- the user can view the total count of pages and their current progress through the results
- the user can navigate to a particular page
- the implementation of offset pagination is relatively trivial

However, the offset pagination is a suboptimal approach for displaying the chat messages due to the following drawbacks:

- not scalable for large datasets because the database must read up to the offset and count rows from the disk only to discard the offset and return the limit rows
- the page window is unreliable when items are written to the database at a high frequency causing duplicate results or even skipping some results

In conclusion, do not use offset pagination to display the chat history.

| Cursor-based Pagination

The cursor-based pagination makes use of an offset (**pointer**) to a particular item in the dataset. The server returns the result set after the provided pointer in the subsequent API requests. The following operations are executed in cursor-based pagination ¹³:

1. A sequential unique column is chosen for pagination
2. The API request will include a parameter known as the cursor parameter to indicate the offset of the results
3. The cursor parameter is used by the server to paginate through the result dataset

The client obtains the cursor (**pointer**) parameter from the server response of the previous API request. I will walk you through an example of the implementation of cursor-based pagination for scrolling through the chat history from the most recent to the oldest chat message. The following SQL will fetch the recent chat messages that will fit the viewport of the client:

SQL



System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

```

4 ORDER BY created_at DESC
5 -- limit = count+1
6 LIMIT %limit;

```

The limit parameter in SQL is set to the count of results to be shown on page + 1. The count of results to be displayed on the page depends on the viewport of the client. The additional record that was queried is not returned in the API response but instead used to identify the value of the next cursor.

The Base64 encoded value of the additional record ID can be returned as the next cursor value to prevent the client from implying the next record and also to enable the server to include supplemental information within the cursor. On top of that, Base64 encoding the cursor provides the flexibility to implement different pagination schemes behind a consistent interface for multiple API endpoints.

The client must provide the value of the cursor in the subsequent API request for pagination¹³. The API response from cursor-based pagination will contain the following data schema:

JavaScript

```

1 {
2   "messages": [{...}],
3   "cursor": "dN315642McETHw" //Base64 encoded next record ID
4 }

```

The following SQL will fetch the recent chat messages before the cursor parameter, which will fit within the viewport of the client:

SQL

```

1 SELECT *
2 FROM chat
3 WHERE channel_id = %channel_id
4 AND created_at <= %cursor
5 ORDER BY created_at DESC
6 -- limit = count+1
7 LIMIT %limit;

```

The limit parameter in SQL is set to the count of results to be displayed on the page + 1 to determine the next cursor. The cursor is set to empty when the client reaches the last page. The following summarizes the benefits of cursor-based pagination¹³:

- scalable for large datasets because the database index is leveraged to prevent a full table scan
- pagination window is not affected when high-frequency writes to the database occur as the next cursor remains the same

There are no solutions, but only tradeoffs. The limitations of cursor-based pagination are the following¹³:

- the cursor must be based on a unique and sequential column in the table
- the client will not be able to navigate to a particular page

The verdict is to use the cursor-based pagination for an efficient display of the chat history on Slack.

| Further system design learning resources

Download my system design playbook for free on newsletter signup:

The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#), and [our Information collection notice](#)

substack

Slack Database

Slack is a read-heavy chat service. Put another way, the typical usage pattern is the client reading the chat messages in a Slack channel.

| Database Schema

The major entities of the relational data schema for Slack are shown in Figure 2.

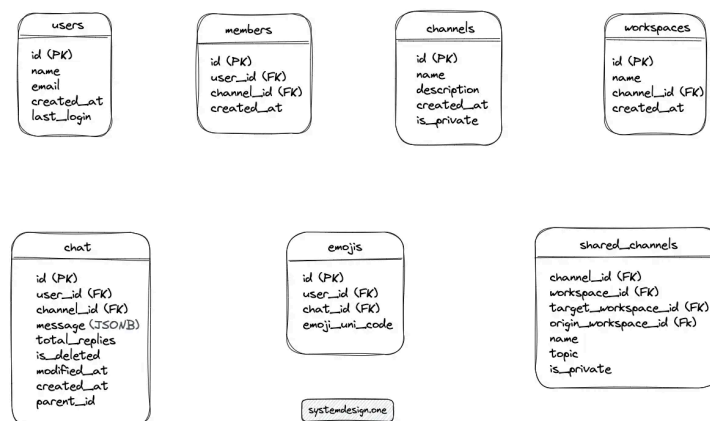


Figure 2: Slack Database schema

A relational database such as MySQL supports JSON data type columns in the relational table. The *chat* table can be used to persist the emojis in a **JSON** data type column instead of generating a dedicated **emojis** table. The write-behind cache pattern can be enforced to store the emojis with high performance and low latency.

A **shared channel** is similar to a normal Slack channel but acts as a connecting bridge between multiple organizations. A single copy of the shared channel data should be persisted and the data access requests must be routed to the database **shard** hosting the channel for scalability and performance. The *shared_channels* table can be used to bridge the workspaces in a shared channel ¹⁴.

| Write a SQL query to fetch the recent 20 chat messages

SQL

```
1 SELECT *
2 FROM chat
3 WHERE channel_id = 4286
4 ORDER BY created_at DESC
5 LIMIT 20;
```

The database index can be initialized to read the chat messages with low latency at the expense of increased storage and elevated latency on write operations.

System Design

The type of database to build the data storage layer for Slack can be decided by inspecting the data types and user traffic patterns. A relational database such as MySQL can be used to build Slack due to the following reasons ⁴:

- mature technology
- the relational data model is a good discipline
- strong support for tooling
- a large talent pool of experienced engineers is available

The relational database should be configured as an eventually consistent data store to favor high availability over strong [consistency](#).

Back of the Envelope Slack

The frequency of clients reading the chat messages is significantly higher than the frequency of clients writing chat messages on Slack. The calculated numbers are approximations. A few helpful tips on [capacity planning](#) during the system design interview are the following:

- 1 million requests/day = 12 requests/second
- round off the numbers for quicker calculations
- write down the units while doing conversions

| Traffic

Slack is a read-heavy chat service. The Daily Active Users (**DAU**) count is 10 million. The number of peak [concurrent](#) users is 7 million according to the historical data. You can assume that a user sends 100 chat messages on average each day.

Description	Value
DAU (write)	1 billion
QPS (write)	12 thousand
read: write	10: 1
QPS (read)	120 thousand

| Storage

The chat messages can be archived and kept in a cold store to reduce storage costs. Besides, chat messages older than a specific time frame should be removed to reduce storage costs. The storage size of each character on a chat message is assumed to be 1 byte. An average chat message is assumed to consume 100 bytes of storage.

Description	Calculation	Total
storage for a day	12k messages/sec * 100 bytes/message * 86400 sec/day	100 GB/day
storage for 5 years	100 GB/day * 5 * 300	150 TB

The replication factor of the storage server should be set to at least three for improved durability and disaster recovery.

| Bandwidth

Ingress is the network traffic that enters the service when chat messages are being written. **Egress** is the network traffic that exits the service when chat messages are being read by the users. The network bandwidth is spread out across the globe depending on the location of the users.

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

Ingress	12k message/sec * 100 byte/message	1.2 MB/sec
Egress	120k message/sec * 100 byte/message	12 MB/sec

The majority of the network traffic is caused by transient events such as [online presence status](#) changes. The metadata of the user profile and chat messages were intentionally skipped during the capacity planning analysis to keep the calculations simple. The performed calculations should give you a rough idea of the [back-of-the-envelope](#) analysis in a system design interview.

| Further system design learning resources

Download my system design playbook for free on newsletter signup:



[The System Design Newsletter](#)

Download my system design playbook on newsletter signup for FREE

By Neo Kim · Over 235,000 subscribers



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#), and [our Information collection notice](#)

substack

Slack Architecture

| Prototyping Slack

Slack can be implemented with a client-server architecture. The chat service identifies the type of event and communicates with the chat database via create, read, update, and delete (**CRUD**) operations ². For instance, the media files should be persisted in object storage, and the links to access the media files can be stored in the chat database. In addition, the chat service returns the JavaScript code for the initial render of the Slack client ³, ¹¹.

The gateway server broadcasts the chat messages to the subscribed clients over the WebSocket channel ¹⁴, ¹¹. The WebSocket channel handles multiple event types such as chat messages, typing indicators, and [user presence status](#) changes.



Figure 3: Slack Prototype

The following operations are executed when the user launches Slack client and publishes a chat message ^{11, 14, 5, 3, 2, 15}:

1. The Slack client executes an HTTP POST request to the web API for user authorization
2. The server sends a snapshot of the workspace and the WebSocket URL to establish a WebSocket channel
3. The client sends a chat message to the chat service over HTTP
4. The chat service persists the chat message in the chat database and delegates the chat message to the subscribed gateway servers over HTTP
5. The gateway server broadcasts the chat message to the subscribed users over WebSocket

The client will establish a connection to the nearest data center using GeoDNS ³. The snapshot makes the user experience seamless when Slack is initially launched by returning the metadata of subscribed Slack channels and the list of users subscribed to the respective Slack channels ⁵. However, the snapshot size increases linearly with the increase in the number of users. The snapshot size can easily grow up to a few dozen megabytes for large organizations resulting in a dreadful latency ^{2, 11, 5}.

An additional **cache** server such as Redis can be provisioned around the chat database for improved read performance. The materialized views of SQL queries can be provided by the cache server to achieve low latency ⁴. The benefits of the Slack prototype architecture are as follows ⁵:

- data model offers a rich real-time client experience at the cost of increased memory usage by the client
- trivial implementation model and horizontally scalable



Figure 4: Database sharding in Slack prototype

The metadata store is a dedicated database containing details about chat database shards. The chat database is the source of truth for chat messages, comments, emojis, and metadata of Slack channels ¹⁶. Every data access operation must query the metadata store to identify the chat database shard before interacting with the respective chat database shard ⁵. The database can be configured in an active-active replication topology across distinct data centers for improved performance, fault tolerance, and scalability ^{4, 14}. The following are the benefits of active-active database replication topology ¹⁶:

- provides high availability through multiple leaders
- high product development velocity with a simple data shard model
- easiness in debugging the customer database
- easy to scale out by adding more database shards

The limitations of the active-active database replication topology are the following ¹⁶:

- scaling limitations on a single host
- inflexible data model
- hot spots and underutilization of shards due to the difficulty in re-sharding
- degraded service availability due to dependency on the database shard and difficulty to configure replicas
- increased operational complexity

The sharding logic should be implemented on the chat service with the current architecture ¹⁶. A channel in Slack is an abstract keyword whose identifier (**ID**) is assigned to an entity such as a user, team, or a regular Slack channel ³. The chat database should be partitioned with *channel_id* as the shard key for scalability and reduced hot spots. A naive approach to partitioning the database is to perform an arbitrary sharding with support for manual re-sharding and automatic re-sharding on the occurrence of a hot spot ⁴.

When the leader instance of the metadata store crashes, an automated failover can be performed to the remaining available leader instance for high availability ⁵. When both leader instances of the metadata store fail simultaneously, the users can continue to use Slack by querying the cache server, but with limited functionality ⁴. The database nodes can be addressed in a configuration file for [service discovery](#) and must be manually replaced for improved availability ⁵.

The [Command and Query Responsibility Segregation \(CQRS\)](#) pattern can be implemented by routing the read requests to the follower database instances and routing the write requests to the leader database instance. The CQRS pattern offers

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

The current architecture is a functional prototype of Slack. However, the current architecture will not satisfy the scalability requirements of Slack.

| Building the Minimum Viable Product for Slack

The requirements for building Slack demand an implementation of [atomic broadcast](#) in distributed systems. The following criteria should be satisfied to implement atomic broadcast:

- validity: all users will eventually receive the message published by a user in a Slack channel
- integrity: a published chat message is received by each user not more than once
- total order: all users receive the chat messages in the identical order

The atomic broadcast in distributed systems is equivalent to consensus¹⁷ and distributed system consensus in general is impossible¹⁸. The key to building Slack is relaxing some constraints to satisfy the system requirements². The constraint that should be dropped is an end-to-end property¹⁹ of the system and depends on the requirements and usage patterns².



Figure 5: Slack; Minimum Viable Product

The [real-time platform](#) is one of the crucial components to build Slack. Both the sender and the receiver (clients) should maintain a WebSocket connection to the [real-time platform](#)²⁰. The real-time WebSocket connection can be used to broadcast chat message updates and initiate API calls to fetch the channel chat history⁵.

The chat messages should be persisted in the chat database before broadcasting to the users for improved durability and fault tolerance. The relational database must be configured to prioritize availability over consistency and support an automatic leader promotion on failure. The relational database produces a statement-based log on every write operation for improved fault tolerance⁴.

The active-active database replication topology comes with a limitation of the possibility of conflicts. The conflicts can either be resolved automatically through application logic or manually through the operator action. The sharding by *channel_id* as the shard key reduces the probability of conflicts as teams cannot overlap⁴.



Figure 6: Scaling the Slack minimum viable product with an edge proxy and snapshot service

The gateway server is a component of the [real-time platform](#) ²⁰. The gateway server is a stateful and in-memory service that maintains the mapping between the *channel_id* and client connections ^{5, 3}. The gateway server also stores some amount of chat history temporarily.

An edge proxy such as [Envoy](#) can be introduced between the gateway server and the client for SSL termination and proxying the WebSocket connections ^{5, 9}. The edge proxy delegates the client requests to the gateway server. [Consistent hashing](#) can be used as the load-balancing algorithm to map the channels to the gateway server ³.

[Hot restarts](#) can be enabled on the edge proxy for scalability, reduced operational complexity, and high availability. Hot restarts prevent dropping any client connections when dynamic changes to the cluster and endpoints occur. The advantages of an edge proxy are the following ⁹:

- enable passive health checking through [outlier detection](#)
- provides [panic routing](#) to handle traffic spikes

The snapshot service is an application-level edge query engine backed by a cache server and stores the bootstrap payload for Slack client ^{5, 4, 14, 11}. The snapshot service will return a slimmed-down version of the bootstrap payload for reduced latency and high performance ¹⁵. The snapshot service offers HTTP API endpoints to the client for fetching data ¹⁵.

The client can query the snapshot service directly to identify the list of users in a particular Slack channel or to fetch the metadata of a particular user ¹¹. On top of that, the snapshot service supports the autocomplete queries on Slack [quick-switcher](#) ¹⁵.

The snapshot service is an application-aware microservice to reduce the server load ². The snapshot service can utilize the [adaptive replacement cache \(ARC\)](#) cache eviction policy on the internal cache for high performance. The snapshot service supports lazy loading of the bootstrap payload when the Slack client is launched for improved latency. The snapshot service comes with the challenge of cache coherency and reconciling business logic.

The client is not allowed to update the snapshot service. The gateway server and chat service can update the snapshot service when new events are received. The in-memory snapshot service should be able to handle approximately 1 million queries per second according to the historical data and load test results from Slack ^{5, 11, 15}.

The snapshot service generates a response to client requests by querying the internal cache. The snapshot for a particular team can be kept up-to-date as long as there is at

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

The snapshot service should be prewarmed by leveraging the publish-subscribe technique offered by the [real-time platform](#) to improve the user experience (latency) of the first user from the specific team.

The snapshot service should store a data object only once in the internal cache and keep references to the particular data object to save memory ¹¹. The snapshot service can subscribe to the relevant Slack channels on the [real-time platform](#) and keep the internal cache updated without duplicates.

The snapshot service provides **just-in-time annotation** by predicting the data objects that the client might query next and pushing the predicted data objects to the clients proactively in the same response to reduce latency by preventing unnecessary network round trips ¹¹. For example, the snapshot service will send the metadata object of an inactive user along with the chat message that mentioned the inactive user because the metadata of the inactive user might not be cached by other users ¹⁵.

The circuit breaker pattern can be implemented for improved fault tolerance of the snapshot service. The snapshot service must reject the incoming traffic when the upstream services indicate an increasing failure rate.



Figure 7: Gateway server; Actor model

The [actor programming model](#) is used to implement the gateway server for improved performance and high throughput. An actor is an extremely lightweight object that can receive chat messages and execute actions to handle the messages using an event handler. A thread will be assigned to an actor when a chat message should be processed. The thread is released once the message is processed and the thread is subsequently assigned to the next actor. Further explanation of the [actor model](#) implementation of the gateway server is included in the [real-time platform](#) article.

The client (mobile device) is allowed to send a chat message such as replying to a Slack notification without establishing a WebSocket connection. The client can make an HTTP request to the chat service for improved performance and low bandwidth ².

The client should [salt](#) the received chat messages to prevent displaying the same message twice. The salt is a random unique token that should be stored with every chat message. The server should not perform deduplication of the chat messages because it is likely that the sender might publish the same chat message twice ².



Figure 8: Slack; Publish-subscribe pattern

A further optimization is to push updates only to the active channels and users in the client's viewport for improved performance and reduced bandwidth usage ¹¹. The clients can subscribe to the events relevant to the current viewport and change subscriptions whenever the user switches to another screen for improved performance ¹⁵.

The publish-subscribe (**pub-sub**) pattern to broadcast events can be built by leveraging the dispatcher service and the endpoint store of the [real-time platform](#). A message broker such as Apache Kafka is not used to build the pub-sub server for reducing operational complexity.

The gateway server subscribes to the endpoint store with all the active channels based on [consistent hashing](#) asynchronously ³. The snapshot service should also subscribe to the relevant channels to receive updates ⁵. The dispatcher queries the endpoint store to identify the list of subscribed gateway servers and snapshot service instances for a particular Slack channel ²⁰.

The subscription workflow of the gateway server and endpoint store in Slack is similar to the subscription workflow in the [real-time platform](#) ²⁰. You can read the related article to explore the subscription workflow in more detail.

| Scaling Slack

Load balancers must be provisioned between different components of [Slack](#) to improve the performance and availability of the system. Object storage such as Amazon Web Services Simple Storage Service (**AWS S3**) can be used to store the media files that are shared in a Slack channel. The [content delivery network \(CDN\)](#) should be used to cache frequently accessed media assets to achieve low latency.



Figure 9: Scaling Slack

The service registry such as the [consul](#) enables service discovery through self-registration of services and automated cluster repair ^{5, 3, 4}. The topology management is automated through the usage of a service registry. The service registry must be queried to identify the service instances available in the system ³.

The job queue is used to defer non-critical tasks such as indexing chat messages to provide chat search functionality ^{2, 14}. The job queue can be provisioned using Redis streams or Apache Kafka ^{11, 4}. The job workers will asynchronously poll the tasks from the job queue ⁴.

The chat messages are persisted in the chat database before being sent to the users over WebSocket ¹⁶. The data storage layer of Slack must be highly available, reliable, and scalable to fulfill the system requirements.

The database shards will exhaust and require frequent re-partitioning in gigantic organizations. [Vitess](#) can be used for partitioning the chat database. Vitess offers a shard topology management service on top of MySQL database for easier horizontal scaling. Vitess allows application code and database queries to remain oblivious to the data distribution among multiple database shards by encapsulation of shard routing logic. Vitess makes it trivial to re-partition a database shard or merge shards according to the evolving system requirements ⁵.

The database can be partitioned with *channel_id* as the shard key to reduce the occurrence of hot spots. The sharding by *channel_id* spreads out the load but introduces the need for the scatter-gather pattern for teams in an organization. The teams are relatively less isolated with the *channel_id* key partitioning approach, so there must be additional performance protective measures implemented against noisy neighbors in the database shards.

The client should be tolerant to partial failures and retry on scatter-gather for improved high availability ⁵. The scatter-gather pattern can be used with Vitess for multiple shards to reduce hot spots. Vitess replaces the leader-leader database replication topology with a single leader and an [orchestrator](#) to execute the automated failover. The following are the benefits of using Vitess to shard the chat database ^{5, 16}:

- flexible sharding through per-table sharding policy
- mature topology management through database self-registration
- supports a single leader with semi-synchronous replication
- reduced operational complexity through an automated failover with the orchestrator
- automatic re-sharding for cluster expansion
- keeps MySQL at the core of development and operations

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

The search database such as Apache Solr should also be partitioned based on *channel_id* as the shard key for scalability ⁴.



Figure 10: Slack; Scaling the snapshot service

The snapshot service must be deployed across multiple edge data centers for scalability and low latency. Team affinity on the snapshot service can be implemented at a data center level by routing all the users in a specific team to the same snapshot service instance. Team affinity is enforced to achieve optimal cache efficiency ¹⁵.

Alternatively, the scatter-gather pattern can be used to improve the cache efficiency of the snapshot service at the cost of elevated latency ¹¹.

An edge proxy can be provisioned in front of the snapshot service to load balance the traffic among snapshot service instances using [consistent hashing](#) using *channel_id* as the key ³. Put another way, [consistent hashing](#) is used to identify the snapshot service instance for a particular Slack channel ⁵.



Figure 11: Slack; Publish workflow of a chat message across multiple data centers

The users of an organization typically do not change their physical location quite often. A dedicated endpoint store could be used to persist the data center ID of each client connection and channel subscription. The published chat messages should be published only to the set of data centers that are in the subscriber list to reduce bandwidth usage.

The cross-data center subscription model can be used to implement Slack to reduce unnecessary bandwidth usage, unlike the broadcast model in the implementation of the [real-time platform](#) ²⁰. Alternatively, the Conflict-free Replicated Data Type (**CRDT**) database can be used to implement cross-data center subscriptions without executing a cross-data center request at the expense of degraded latency. The endpoint store will contain the following data model ²⁰:

☰ Data Model of Endpoint Store

```
{ channel_id : set (data_center_id) }
```

The following operations are performed by Slack to publish a chat message across multiple data centers ^{20, 3}:

1. The client makes an HTTP request to the web API of the chat service to publish the message
2. The chat service persists the message in the chat database and delegates the message to the dispatcher service
3. The chat service updates the snapshot service
4. The dispatcher queries the local endpoint store to identify the data centers of the subscribed clients
5. The dispatcher broadcasts the chat message only to the set of subscribed peer data centers over the HTTP
6. The dispatcher in the peer data center queries the local endpoint store to identify the list of subscribed gateway servers
7. The gateway server queries the local in-memory subscription store to identify the list of subscribed clients and fans out the chat message over WebSocket

| Further system design learning resources

Download my system design playbook for free on newsletter signup:

The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#) and [our Information collection notice](#)

substack

Slack Architecture Deep Dive

The microservice architecture can be used for service decomposition and scalability. The service mesh is used for [service discovery](#) as the system scales out ⁵.



Figure 12: Slack; Timeline diagram for chat message send-receive workflow

The clients can publish chat messages without establishing a WebSocket connection at the cost of an additional network hop between the clients. The latency of the system can be reduced by deferring the expensive computations using the job queue ^{2, 4}. The unread notifications and user reactions in Slack can be implemented using a [distributed counter](#) at the expense of increased operational complexity.

The push notifications in Slack can be enabled through third-party service integrations. The push notification allows notifying an inactive user, for example, when the inactive user is mentioned in a chat message. Some of the extremely popular push notification services are the Apple push notification service and Firebase cloud messaging. The [bloom filter](#) data structure can be employed to prevent the duplication of notifications to the same user in a space-efficient approach ²¹.

The client can store the timestamp of the last read message to identify the offset of messages to be retrieved. However, this approach will highly likely result in an erratic user experience due to the usage of multiple devices by the same user. As an

| Presence Status

The events such as a user reaction to a message, bookmarking, or a member joining a channel follow a similar workflow as publishing a chat message. The transient events such as typing indicators and presence status changes can be sent through the WebSocket channel instead of communicating with the chat service ³.

The majority of the events occurring on Slack are user presence status change ^{4, 6}. The publish-subscribe technique can be used for an efficient implementation of the user presence status ¹¹. The Slack client should display the presence status only for the subset of users that are currently visible on the viewport of the client for improved performance ³. The [real-time presence platform](#) article describes in detail the implementation of the user presence status feature ²³.

| Message Ordering

The server timestamp can be used as the chat message ID to order the messages in chronological order. However, the message ordering with the server timestamp will yield inaccurate results in a distributed system due to the occurrence of the clock skew and network delays. On the other hand, the client timestamp cannot be used due to the possibility of an incorrect time set and difficulty in synchronizing time across clients.

An alternative approach is to use a unique sequence number generator such as [Twitter Snowflake](#) for ordering the chat messages. However, this approach also suffers from the clock skew problem in a distributed system and will not result in accurate message ordering.

The precise timestamp of the messages is not crucial but only the causality of the messages is important for proper message ordering. As a result, the optimal approach is to use a [logical clock](#) such as the Lamport clock or a Vector clock for message ordering. The logical clock is a mechanism for capturing chronological and causal relationships within a distributed system. The key concept is to annotate every chat message request with an incremented count of the previous chat message response received by the client.

| Low Latency

The client persists the timestamp of the last synchronization with the snapshot service to enable incremental updates of the snapshot ^{2, 11}. The service workers in the web client can be used to fetch only the incremental updates from the snapshot service ²². The session objects can be loaded asynchronously in parallel by the client to maintain low latency. The hashcode of the media file URLs should be shared with the clients for reduced bandwidth ¹¹.

The mobile clients should load only a single Slack channel and load other channels in the background to accomplish low latency. The JSON events can be replaced with Thrift events for ease of maintenance and improved performance ¹¹. The snapshot size can be further reduced by removing some of the data fields from the initial payload.

The cache server can hold the unread messages and their count for rapid retrieval. The mobile client should poll the API endpoint to keep the sidebar updated ²⁴.



Figure 13: Slack; Smart edge cache

An edge cache can be used to reduce the network round trip to improve the latency. The libchat is an internal service used by different components to query their local cache.

The network round-trip time on the desktop and mobile clients can be reduced by querying the local cache in the client via the libchat service. The libchat service and local cache can run as distinct processes on the desktop client for improved performance⁷.

The users of the web client often have stable, and high bandwidth internet connections like the office internet or wired ethernet. The libslack service cannot be initialized on a web client due to the difficulty in injecting native code on the web browser. Therefore, the web client can execute requests to the snapshot service provisioned on the edge data centers⁷.

The communication between the components of Slack occurs through the libchat service. This method provides an opportunity to experiment with different network protocols and wire formats with changes to a single codebase⁷.

| High Availability

An organization reconnecting to the Slack network after an outage could cause a thundering herd⁴. There should be multiple countermeasures in place to tackle the thundering herd problem. The in-memory snapshot service can reduce the server load and offers a straightforward approach to mitigate the thundering herd problem^{2, 15}.

Jitter must be added to client connection request time intervals to spread out the load and reduce the thundering herd problem. The partitioning of stateful services of Slack with `channel_id` as the shard key can also prevent a thundering herd⁵.

An active-active load balancer should be provisioned to handle traffic spikes for high availability. Rate limiting should be configured on the services and API endpoints to attain high availability and fault tolerance. The requests can be dropped when a particular service instance reaches its threshold for memory, concurrent connections, or throughput^{11, 4}. The rate limiter will also alleviate the thundering herd problem⁴.

The load testing frameworks such as `locust` can be used for identifying the service limits to prevent cascading failures and improve fault tolerance¹¹. There should be cluster orchestration configured to self-heal for high availability⁵.

The hot spots on the system can be handled by alerting and provisioning high-end hardware for critical services⁴. A dedicated monitoring service should be provisioned

System Design

[Join Newsletter](#) [Archive](#) [About](#) | [Q](#) [🔍](#)

| Fault Tolerance

A draining mechanism should be implemented to route the users to a working data center when the data center fails for improved fault tolerance and high availability³. A dead letter queue can be provisioned to log the errors while writing to the chat database.

The gateway server maintains an in-memory queue of uncommitted sends for transient events. The events should be replayed when a gateway server fails. The gateway server should automatically retry when the chat service is unavailable for improved fault tolerance². The depth of the in-memory queue must be monitored for asserting the system's health⁴.

| Scalability

The developer's productivity can be increased by provisioning a remote development environment²⁵. The infrastructure can leverage services offered by the hyperscalers such as AWS for rapid growth and scalability. The resources can be provisioned and disaster recovery scenarios can be practiced with ease on the hyperscaler infrastructure²⁶.

| Client Support

The desktop Slack client can be built with [ElectronJS](#) for cross-platform support. The web client can be created with any modern JavaScript framework such as [ReactJS](#)²². The mobile client can be built with either [React Native](#) or native mobile technologies.

| Security

Security issues must be proactively detected through static code analysis. Static code analysis is a tool that inspects the code for security vulnerabilities and insecure programming practices without code execution. The static code analysis is based on a list of security rules. Some of the popular static code analysis tools are [checkmarx](#) and [semgrep](#)²⁷.

Summary

The key takeaway from building Slack is that optimality is contingent and should be adapted with growth. It is crucial to identify the end-to-end part of the problem to build a distributed system².

What to learn next?

Download my system design playbook for free on newsletter signup:

The System Design Newsletter

Download my system design playbook on newsletter signup for FREE

By Neo Kim 🏆 · Over 235,000 subscribers

amanpunetha08@gmail.com



By subscribing you agree to [Substack's Terms of Use](#), [our Privacy Policy](#), and [our Information collection notice](#)

substack

License

CC BY-NC-ND 4.0: This license allows reusers to copy and distribute the content in this article in any medium or format in unadapted form only, for noncommercial purposes, and only so long as attribution is given to the creator. The original article must be backlinked.

References

1. [What is Slack?](#), slack.com ↩
2. Keith Adams, [Scaling Slack](#) (2020), gotoams.nl ↩
3. Sameera Thangudu, [Real-time Messaging](#) (2023), slack.engineering ↩
4. Keith Adams, [How Slack Works](#) (2016), infoq.com ↩
5. Mike Demmer, [Scaling Slack- The Good, the Unexpected, and the Road Ahead](#) (2018), infoq.com ↩
6. Julia Grace, [Scaling Infrastructure Engineering at Slack](#) (2019), infoq.com ↩
7. Keith Adams, Mike Fleming, Haim Grosman, [Making Slack Feel Like Slack](#) (2017), slack.engineering ↩
8. Brenda Jin, [Evolving the Slack API](#) (2018), slack.engineering ↩
9. Ariane van der Steldt, Radha Kumari, [Migrating Millions of Concurrent Websockets to Envoy](#) (2021), slack.engineering ↩
10. Allan Denis, [Will WebSocket survive HTTP/2?](#) (2016), infoq.com ↩
11. Bing Wei, [Scaling Slack](#) (2018), infoq.com ↩
12. Saurabh Sahni, Taylor Singletary, [How We Design Our APIs at Slack](#) (2022), slack.engineering ↩
13. Michael Hahn, [Evolving API Pagination at Slack](#) (2018), slack.engineering ↩
14. Yingyu Sun, Mike Demmer, [How Slack Built Shared Channels](#) (2020), slack.engineering ↩
15. Bing Wei, [Flannel: An Application-Level Edge Cache to Make Slack Scale](#) (2018), slack.engineering ↩
16. Guido Iaquinti et al., [Scaling Datastores at Slack with Vitess](#) (2021), slack.engineering ↩
17. Tushar Deepak Chandra et al., [Unreliable failure detectors for reliable distributed systems](#) (1996), dl.acm.org ↩
18. Michael J. Fischer et al., [Impossibility of Distributed Consensus with One Faulty Process](#) (1985), groups.csail.mit.edu ↩
19. J.H. Saltzer et al., [End-to-End Arguments in System Design](#), web.mit.edu ↩
20. Neo Kim, [Live Comment System Design](#) (2023), systemdesign.one ↩
21. Suresh Kondamudi, [How To Remove Duplicates In A Large Dataset Reducing Memory Requirements By 99%](#) (2016), highscalability.com ↩
22. [Slack Frontend Architecture with Anuj Nair](#) (2020), softwareengineeringdaily.com ↩

System Design

24. Johnny Rodgers et al., [Reducing Slack’s memory footprint](#) (2017), slack.engineering ↩

25. Sylvester George, [Remote Development at Slack](#) (2022), slack.engineering ↩

26. [Slack Case Study](#) (2015), aws.amazon.com ↩

27. [How Two Interns Are Helping Secure Millions of Lines of Code](#) (2022), slack.engineering ↩

Updated on 2023-05-28

CC BY-NC 4.0



[Back](#) | [Home](#)

[◀ Back of the Envelope](#)

[What Is Service Discovery? ▶](#)



Sponsored

Mr. Bala's Powerful Intraday Strategy Revealed – No More Guesswork

TradeWise

Learn More

Make Massive Side Income with this Method of Intraday Trading

TradeWise

Learn More

Join new Free to Play WWII MMO War Thunder

War Thunder

Play Now

World's Tallest Temple for Krishna in Progress

Vrindavan Chandrodaya Mandir

Click Here

War Thunder - Register now for free and play against over 75 Million real Players

War Thunder

Karnataka: Support World's Tallest Religious Monument

Vrindavan Chandrodaya Mandir

Click Here

by Taboola



Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

🤍 10

Share

BestNewestOldest



Kyle

3 years ago

Great article, thanks for the contribution!

10Reply🔗

—

🚩



Manikandan Gunasekaran

2 years ago

One of the finest articles, thanks to @NK

00Reply🔗

—

🚩



Masoud Sanaei

3 years ago

Can you tell me what software was used to draw the diagrams in this article?

00Reply🔗

—

🚩



NK

Author

➔ Masoud Sanaei

3 years ago

The diagrams were created with excalidraw.

10Reply🔗

—

🚩



Masoud Sanaei

➔ NK

3 years ago

Thank you.

10Reply🔗

—

🚩

Subscribe Privacy Do Not Sell My Data